

Aether Vision

YOLOv11 Object Detection Web Application

Created by: Ashish-Arjit

Date: May 30, 2026

Repository: github.com/Ashish-Arjit/object_detection_demo_1

1. Executive Summary

Aether Vision is a premium web application developed to perform real-time, serverless and server-side object detection. The application utilizes the state-of-the-art YOLOv11 deep learning model to process image, webcam, and video streams. The design emphasizes visual excellence with a custom glassmorphic dark-theme dashboard.

To maximize compatibility, the application automatically falls back to client-side ONNX Runtime Web execution (using WebAssembly) if a local Python backend is unavailable. This allows 100% serverless hosting on static spaces like GitHub Pages, bringing deep learning capabilities directly to user web browsers.

2. System Architecture

The project has been engineered with a hybrid dual-mode system architecture. This allows the app to dynamically transition between backend-driven processing and client-side local WebAssembly processing depending on the deployment hosting environment.

Mode A: Python Flask Backend (Server-Side)

In local environments, Python executes the YOLOv11 model using PyTorch. The client sends base64 images via REST APIs. Flask decodes the image, runs the inference prediction, draws bounding boxes and labels using OpenCV, and returns the modified image. Video frames are compiled asynchronously on a background worker thread using Python threading to prevent server locks.

Mode B: ONNX Runtime Web (Serverless Client-Side)

When hosted on static servers (like GitHub Pages), the app switches to serverless mode. The client browser downloads the 10.2 MB pre-trained ONNX model (yolo11n.onnx) and executes it locally using WebAssembly. The entire preprocessing (resizing to 640x640, channel-first CHW format float32 arrays), model execution, and postprocessing (Non-Maximum Suppression and IoU overlap removal) run inside browser JavaScript. No remote server queries are made.

3. YOLOv11 & COCO Dataset Details

The application utilizes YOLO11 (You Only Look Once), the latest generation computer vision engine developed by Ultralytics. The nano configuration model (yolo11n) was chosen to provide the lowest possible execution latency on typical consumer devices.

The model is pre-trained on the COCO (Common Objects in Context) dataset. This dataset is a massive benchmarking database containing over 330,000 images and 1.5 million labeled instances across 80 different category classes (e.g. people, cars, animals, kitchenware). Because the model weights (yolo11n.pt and yolo11n.onnx) hold the pre-trained neural network weights, the app can perform out-of-the-box predictions without needing to load any raw training dataset.

4. Core Features

- Visual Dashboard: Shows running metrics (total detections, webcam FPS, inference latencies) and displays class distributions on custom bar charts (Chart.js).
- Image comparative slider: Split uploader allowing comparative slider analysis (before vs after).
- Bounding box crops: Extracts cropped bounding boxes directly on the browser canvas without server queries.
- Dynamic Class Filters: Lets users search classes and toggle boxes on/off in real-time.
- Real-time Webcam: Feeds camera frames to the ONNX / Flask engine at 10 FPS and overlays boxes.
- Trash Reset Button: Floating uploader button in the Image Studio allowing quick workspace resets.

5. File Structure

```
+-- app.py                # Python Flask Server & API routes
+-- requirements.txt      # Backend dependencies
+-- yolo11n.pt            # PyTorch model weights
+-- yolo11n.onnx          # Exported ONNX model weights (10.2 MB)
+-- .gitignore            # Excludes virtual environments and temp folders
+-- README.md             # GitHub project markdown guide
+-- templates/
|   +-- index.html        # Frontend page dashboard layout
+-- static/
|   +-- css/
|       +-- style.css     # Translucent CSS styling sheet
+-- js/
    +-- app.js            # Main client script with Web-ONNX engine
```